HDOC Day-56

First Negative Integer in Every Sliding Window Using Pointers in C

Problem Statement

Read a positive array size, a variable-length integer array and a window size k. Pass the array to first_negative() using its address. For each contiguous window of k elements, print its first negative integer, or 0 when none exists. Print -1 for an invalid k. Use nested loops and pointer access.

Algorithm

1. Start the program and read the array size. Reject non-positive sizes and sizes above 10000.
2. Declare a variable-length integer array arr[size] and read its elements.
3. Read integer k and pass arr by address, along with size and k, to first_negative().
4. If k is not in the range 1 to size, print -1 and return from the function.
5. For each starting index i from 0 through size-k, examine the window arr[i] to arr[i+k-1].
6. Using an inner loop, scan the window from left to right and stop at the first negative value.
7. If no negative value is found in the window, use 0 as its result.
8. Print the results in window order separated by spaces; then stop.

Test Case Table

TC	Size	Array elements	k	Expected output	Purpose
1	8	12 -1 -7 8 -15 30 16 28	3	-1 -1 -7 -15 -15 0	Professor example; mixed signs
2	5	1 2 3 4 5	2	0 0 0 0	All positive
3	5	-4 -2 -8 -1 -6	3	-4 -2 -8	All negative
4	4	3 -2 5 -1	1	0 -2 0 -1	Smallest valid k
5	4	3 -2 5 -1	4	-2	k equals array size
6	3	1 2 3	4	-1	k greater than size
7	3	1 -2 3	0	-1	k is zero
8	3	1 -2 3	-1	-1	Negative k
9	1	-9	1	-9	Single negative element
10	4	0 0 0 0	2	0 0 0	All zeros

C Program

```
#include <stdio.h>

void first_negative(const int *arr, int size, int k)
{
    int i, j, result;

    if (k <= 0 || k > size)
```

```
    {
        printf("-1\n");
        return;
    }

    printf("First negative in each window: ");
    for (i = 0; i <= size - k; i++)
    {
        result = 0;
        for (j = i; j < i + k; j++)
        {
            if (*(arr + j) < 0)
            {
                result = *(arr + j);
                break;
            }
        }
        if (i > 0)
            printf(" ");
        printf("%d", result);
    }
    printf("\n");
}

int main(void)
{
    int size, k, i;
    printf("Enter array size: ");
    if (scanf("%d", &size) != 1 || size <= 0 || size > 10000)
    {
        printf("Invalid size. Enter a positive integer up to 10000.\n");
        return 1;
    }

    int arr[size];
    printf("Enter %d array elements:\n", size);
    for (i = 0; i < size; i++)
    {
        if (scanf("%d", arr + i) != 1)
        {
            printf("Invalid array element.\n");
            return 1;
        }
    }
    printf("Enter k: ");
    if (scanf("%d", &k) != 1)
    {
        printf("Invalid k.\n");
        return 1;
    }
    first_negative(arr, size, k);
    return 0;
}
```

Compilation and Execution Commands

```
gcc hdoc56.c -o hdoc56
./hdoc56
```

Screenshot Spaces

Compilation and Execution

```
┌──(kali㉿kali)-[~/Desktop]
└─$ vi hdoc56.c

┌──(kali㉿kali)-[~/Desktop]
└─$ gcc hdoc56.c -o hdoc56
```

Test Case 1

Input: size = 8; array = 12 -1 -7 8 -15 30 16 28; k = 3. Expected result: -1 -1 -7 -15 -15 0. (Professor example; mixed signs)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc56
Enter array size: 8
Enter 8 array elements:
12 -1 -7 8 -15 30 16 28
Enter k: 3
First negative in each window: -1 -1 -7 -15 -15 0
```

Test Case 2

Input: size = 5; array = 1 2 3 4 5; k = 2. Expected result: 0 0 0 0. (All positive)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc56
Enter array size: 5
Enter 5 array elements:
1 2 3 4 5
Enter k: 2
First negative in each window: 0 0 0 0
```

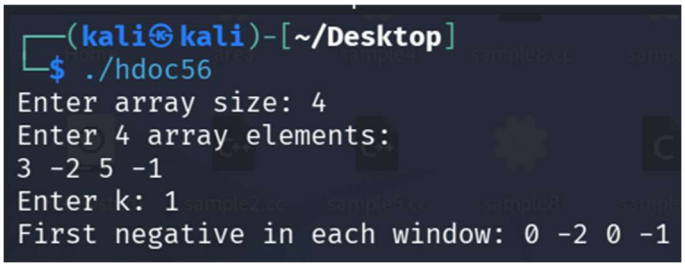
Test Case 3

Input: size = 5; array = -4 -2 -8 -1 -6; k = 3. Expected result: -4 -2 -8. (All negative)

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc56
Enter array size: 5
Enter 5 array elements:
-4 -2 -8 -1 -6
Enter k: 3
First negative in each window: -4 -2 -8
```

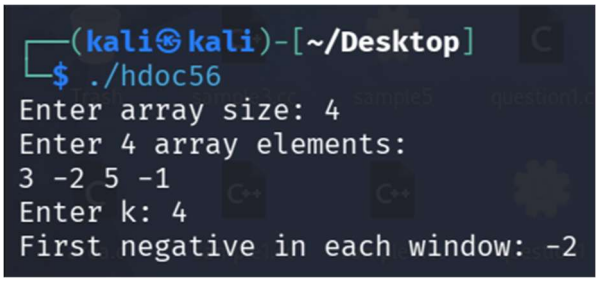
Test Case 4

Input: size = 4; array = 3 -2 5 -1; k = 1. Expected result: 0 -2 0 -1. (Smallest valid k)



Test Case 5

Input: size = 4; array = 3 -2 5 -1; k = 4. Expected result: -2. (k equals array size)



Test Case 6

Input: size = 3; array = 1 2 3; k = 4. Expected result: -1. (k greater than size)



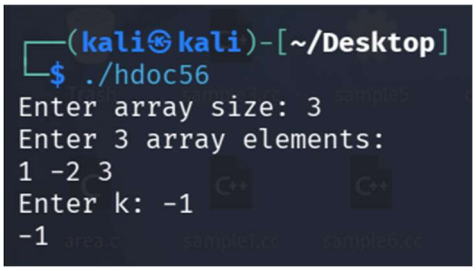
Test Case 7

Input: size = 3; array = 1 -2 3; k = 0. Expected result: -1. (k is zero)



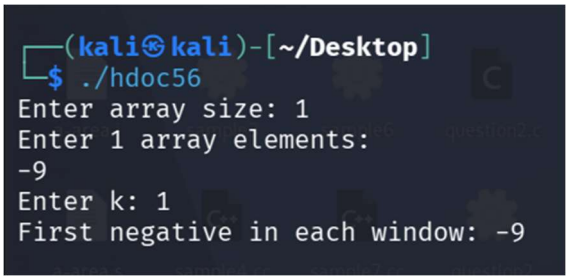
Test Case 8

Input: size = 3; array = 1 -2 3; k = -1. Expected result: -1. (Negative k)



Test Case 9

Input: size = 1; array = -9; k = 1. Expected result: -9. (Single negative element)



Test Case 10

Input: size = 4; array = 0 0 0 0; k = 2. Expected result: 0 0 0. (All zeros)

